

Plan sobresaliente · rama dev llama.cpp

Inspiración Zammad-MCP (tools tickets/users/orgs + resources + prompts), tanss-mcp-server (237 tools, el volumen impresiona), helpdesk-mcp (FastAPI + MCP sobre tickets). Adaptado a Soporte_Tecnico2 con llama.cpp local.

Fase A · MCP server real (Python, FastMCP)

Paquete backend-fastapi/app/mcp_soporte/ (nuevo, no toca lo existente):

- Transporte stdio (Claude Desktop) + HTTP Streamable (:8001... no, :8090) con api-key.
- Auth: recibe el JWT del técnico y lo reenvía a la API (cero bypass de roles).
- Catálogo (nombres estilo Zammad-MCP):

```
| # | Tool | Lee/escribe | ---|---| | 1 | soporte_buscar_atenciones | R | | 2 | soporte_get_atencion | R | | 3 | soporte_crear_borrador (no guarda, devuelve preview) | R | | 4 | soporte_confirmar_creacion (requiere confirm:true + JWT) | W | | 5 | soporte_actualizar_atencion (con confirm) | W | | 6 | soporte_reclasificar (área/grupo, con confirm) | W | | 7 | soporte_stats / soporte_stats_tecnico | R | | 8 | soporte_arbol_jerarquia / soporte_areas | R | | 9 | soporte_usuarios / soporte_horarios | R | | 10 | soporte_ia_preguntar (RAG + guardrails, ya existe) | R | | 11 | soporte_ia_calificar / soporte_ia_promover (jefe) | W | | 12 | soporte_ia_evaluar (batería 10, latencia+fuelle) | R |
```

- Resources: soporte://atencion/{id}, soporte://usuario/{id}.
- Prompts: resumir_ticket, informe_tecnico, borrador_respuesta.
- Regla de oro: toda escritura exige confirm:true explícito + colaborador=Wilmercito (id 14).
- DoD-A: npx @modelcontextprotocol/inspector lista 12+ tools; crear borrador · confirmar · visible en lista con colaborador Wilmercito; test e2e en VM.

Fase B · Poderes en el chat (sobre Fase A)

- El chat propone borrador ("Voy a crear: ¿confirmas[Sí]") · llama a soporte_confirmar_creacion con el JWT de la sesión. Sin 'sí' no hay escritura.
- Informes de jefe en chat: informe_tecnico usa soporte_stats_tecnico + resumen LLM.
- DoD-B: Diego crea una atención real desde el chat; auditoría muestra autor Diego colaborador Wilmercito.

Fase C · PyTorch: dónde ayuda (respuesta a la pregunta)

PyTorch **ya está** en la VM (CPU): es el motor bajo sentence-transformers (embeddings) y el cross-encoder (rerank). llama.cpp, en cambio, es C++/CUDA puro y **no** usa PyTorch (por eso rinde en tu 2060).

```
| Uso | Estado | ---|---| | Embeddings + rerank (inferencia CPU) | · activo | | Clasificador de intents (el actual, por similitud) | · activo | | Clasificador entrenado (Linear head sobre embeddings, 5 min en CPU) | · propuesto: más robusto que umbrales fijos | | Fine-tuning LoRA Qwen2.5-3B (QLoRA nocturno, ver fine-tuning-lora.md) | · planificado | | Entrenar embeddings propios del dominio | · futuro (overkill hoy) |
```

DoD-C: clasificador entrenado con >95% en batería de 30 frases; LoRA evaluado sin regresiones (ver fine-tuning-lora.md §5).

Orden sugerido

A (2·3 sesiones) · B (1·2) · C por partes. Cada fase commitea docs + tests e2e en VM.

Implementado 2026-09-25 (Fase A+B + observabilidad)

- LogIA (pregunta, fuente, rechazado, ms) en cada /ia/preguntar; GET /ia/resumen (jefe): indexadas, preguntas/hora, rechazos/hora, p95, propuestas pendientes.
- PropuestaIA (proponente, tipo, payload, estado, atención, revisor): el chat propone el jefe aprueba/ejecuta desde /asistente. FK a Atenciones protege la auditoría (no se puede borrar una atención ejecutada sin borrar antes la propuesta).
- Panel /asistente: huella de salud + propuestas + reindex + curar conocimiento.